Airline Management System

Object-Oriented Programming (OOP) — Semester Project

C++ APPLICATION

OOP CONCEPTS

Introduction

The **Airline Management System** is a C++ application built to manage real-world airline operations. It demonstrates how Object-Oriented Programming concepts can be applied to solve practical problems.

✈️ Flights

Add, remove, and search flights

👤 Passengers

Register and manage passenger records

🎫 Bookings

Book and cancel tickets with loyalty points

📊 Reports

Generate occupancy and revenue reports

Project Objectives

This project is designed to achieve six core goals that reflect real airline management needs.

1

Flight Management

Maintain a complete database of domestic, international, and charter flights.

2

Passenger Records

Store and retrieve passenger information with booking history.

3

Ticket Operations

Handle booking, cancellation, and seat assignment automatically.

4

Reports

Generate occupancy and revenue reports for airline analysis.

5

Data Persistence

Save and load all records using file handling.

6

OOP Implementation

Demonstrate encapsulation, inheritance, polymorphism, and abstraction.

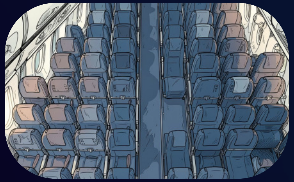
Key Features

Flight Management

- Add, remove, and search flights
- Supports Domestic, International, and Charter types

Passenger Management

- Register and remove passengers
- View complete booking history



Booking System

- Automated seat assignment
- Ticket cancellation with refunds
- Loyalty points on bookings

Reports

- Flight occupancy report
- Revenue summary report

Class Structure

The system is built around three abstract base classes — **Flight**, **Passenger**, and **Airline** — with derived subclasses for each category. The **Ticket** class ties everything together.

Abstract classes enforce a consistent interface while allowing each subclass to implement its own behavior through polymorphism.

OOP Concepts Implemented

Encapsulation

Data is protected using `private` and `protected` members, exposing only what's necessary.

Inheritance

Child classes (e.g., `DomesticFlight`) inherit properties from parent abstract classes.

Polymorphism

Virtual functions allow different flight and passenger types to behave uniquely.

Abstraction

Abstract classes hide implementation details, exposing only the essential interface.

Advanced C++ Features

Operator Overloading

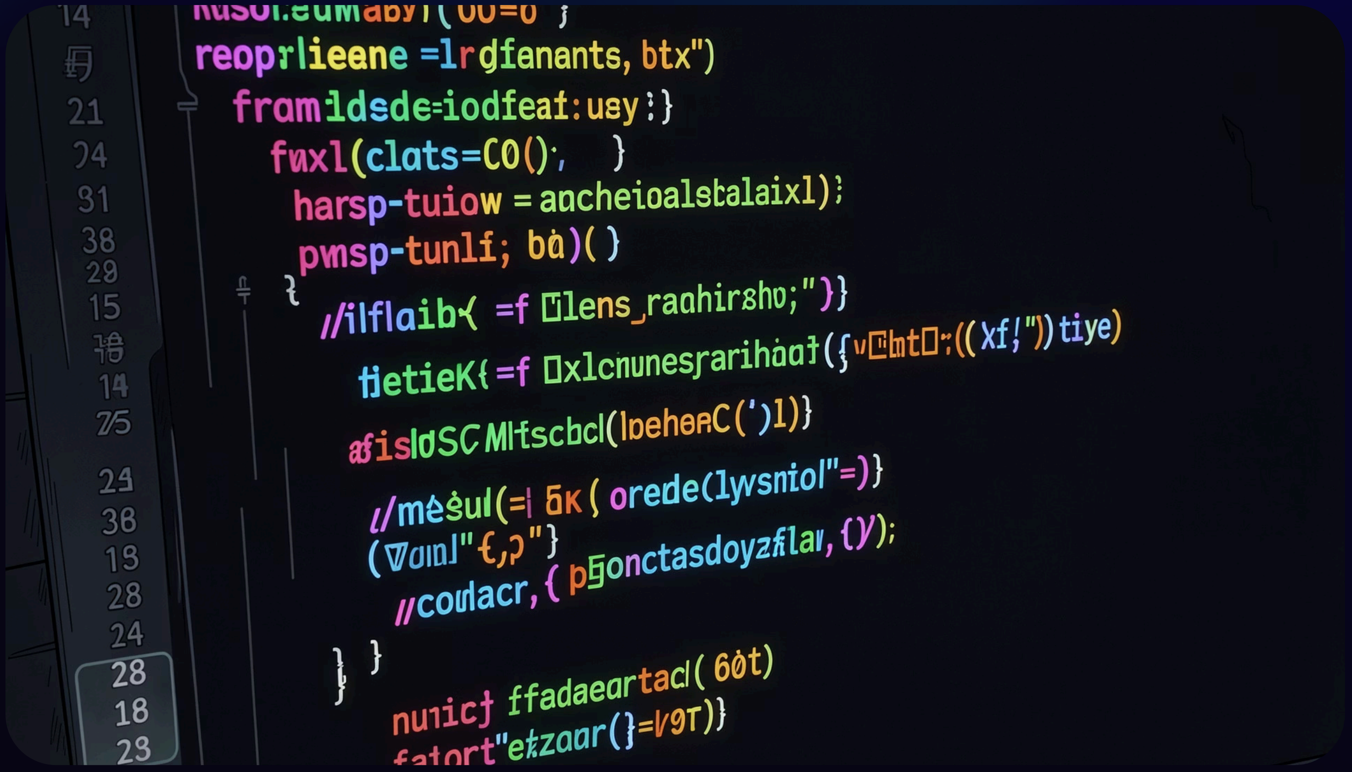
Custom `<<` operators display Flights, Passengers, and Tickets cleanly.

Templates

A generic search function finds records across different data types.

Exception Handling

Custom exceptions: `FlightFullException` and `InvalidCancelException` handle error cases gracefully.



Booking & Cancellation Flow

🎫 Booking Process

01

Passenger enters ID

02

Select flight number

03

System checks seat availability

04

Seat assigned automatically

05

Ticket generated

06

Loyalty points awarded

❌ Cancellation Process

01

Enter Ticket ID

02

System verifies ticket

03

Status set to CANCELLED

04

Seat released

05

Refund calculated

Refund Policy: Economy → 50% · Business → 75% · First Class → 90%

Data Storage & Sample Output

📁 File Handling

All data is persisted in `airline_data.txt`, ensuring records survive program restarts.

- Flight records
- Passenger records
- Ticket records
- Seat information

🖥️ Sample Output

Flight

SK101 → Karachi to Lahore

Passenger

Ali Ahmed

Ticket

ID: TKT1 · Seat: A1 · Fare: Rs. 5500 · Status: ACTIVE

Conclusion

The Airline Management System successfully brings together core and advanced C++ concepts into a cohesive, real-world application.

OOP Mastery

Encapsulation, Inheritance, Polymorphism, Abstraction

Advanced C++

Templates, Operator Overloading, Exception Handling

Real-World Logic

Booking, cancellation, refunds, and data persistence

This project provides a strong practical implementation of C++ programming concepts — ready for real-world airline operations.